Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №20

Дисциплина: Основы программирования на Python

Тема: Контейнеризация, CI/CD и эксплуатация Python-приложения

Вариант 2: Библиотека

Цель работы

Получить практические навыки подготовки Python-приложения к развёртыванию и эксплуатации. В процессе выполнения работы необходимо освоить контейнеризацию приложений, создание Dockerfile, работу с образами и контейнерами, настройку Docker Compose, работу с переменными окружения, управление конфигурацией, подключение базы данных, настройку проверки работоспособности, управление зависимостями сервисов, автоматическую сборку проекта и автоматический запуск тестов, основы CI/CD с использованием GitHub Actions и анализ логов.

Постановка индивидуального задания (Вариант 2. Библиотека)

Контейнеризировать приложение библиотеки и базу данных PostgreSQL. Система должна состоять из Python-приложения и базы данных, запускаемых через Docker Compose, содержать отдельный контейнер для приложения, автоматический запуск тестов и CI-конвейер. Дополнительно требуется добавить проверку работоспособности базы данных и обеспечить запуск приложения только после её готовности.

Краткие теоретические сведения

Контейнеризация — способ упаковать приложение вместе с его зависимостями в изолированную среду. Образ (image) представляет собой

неизменяемый шаблон, а контейнер — запущенный экземпляр образа. В отличие от виртуальной машины, контейнер использует ядро хоста и запускается за секунды, занимая меньше ресурсов.

Dockerfile описывает сборку образа последовательностью инструкций: FROM задаёт базовый образ, WORKDIR — рабочий каталог, COPY переносит файлы, RUN выполняет команды при сборке, ENV задаёт переменные окружения, USER переключает пользователя, EXPOSE объявляет порт, CMD задаёт команду запуска. Многоэтапная сборка позволяет установить зависимости в отдельном этапе и перенести в итоговый образ только готовое окружение, что уменьшает размер и поверхность атаки.

Docker Compose описывает многоконтейнерное приложение: сервисы, образы, переменные окружения, тома для сохранения данных, сети для взаимодействия по именам сервисов и проверки работоспособности healthcheck. Условие depends_on с параметром service_healthy задерживает запуск зависимого контейнера до готовности базы данных.

Конфигурация выносится в переменные окружения, чтобы один и тот же образ работал в разных средах. Health check — периодическая проверка, отвечает ли приложение или база на служебный запрос; при неудаче контейнер помечается как нездоровый. CI/CD — практика автоматического выполнения проверок и сборки при каждом изменении кода; GitHub Actions описывает конвейер в YAML-файле и состоит из задач (jobs), выполняемых на виртуальных машинах.

Исходное приложение и его архитектура

Контейнеризуется приложение библиотеки, разработанное в предыдущих работах: REST API на FastAPI с сущностями Author, Book и Loan, слоями маршрутов, сервисов и репозиториев, поддержкой SQLAlchemy и возможностью работать как с PostgreSQL, так и с SQLite для локальных тестов.

Компонент	Назначение
src/main.py	Фабрика приложения create_app(), lifespan с инициализацией базы, маршрут /health
src/config.py	Настройки на основе pydantic-settings: параметры базы, уровень логирования, секретный ключ
src/database.py	Создание движка SQLAlchemy, фабрика сессий, инициализация схемы
src/api/	Маршруты книг, авторов, выдач и проверки работоспособности
src/logging_config.py	Единая настройка журналирования с уровнем из переменной окружения
tests/	Модульные и интеграционные тесты приложения
Dockerfile, docker-compose.yml	Сборка образа и описание многоконтейнерной системы
.github/workflows/ci.yml	Конвейер непрерывной интеграции: линтер, тесты с покрытием, сборка образа

Dockerfile и его инструкции

Сборка выполнена многоэтапной. На первом этапе builder на основе образа python:3.12-slim создаётся виртуальное окружение и устанавливаются зависимости из requirements.txt без кэша пакетов. На втором этапе runtime в чистый образ копируется только готовое окружение и исходный код, что исключает из итогового образа компиляторы и временные файлы.

Инструкция	Назначение в проекте
FROM python:3.12-slim AS builder	Первый этап сборки: установка зависимостей
ENV PYTHONDONTWRITEBYTECODE, PYTHONUNBUFFERED	Отключение файлов .pyc и буферизации вывода для корректных логов
WORKDIR /build	Рабочий каталог этапа сборки
COPY requirements.txt ., RUN python -m venv	Создание виртуального окружения и установка зависимостей одной инструкцией RUN
FROM python:3.12-slim AS runtime	Второй этап: минимальный образ для запуска
COPY --from=builder /opt/venv /opt/venv	Перенос готового окружения между этапами
RUN groupadd, useradd, USER appuser	Запуск от непривилегированного пользователя
COPY --chown=appuser:appuser . .	Копирование кода с правами рабочего пользователя
EXPOSE 8000	Документирование порта приложения
CMD gunicorn -k uvicorn.workers.UvicornWorker	Запуск двух рабочих процессов под gunicorn

Листинг 20.1 — Многоэтапный Dockerfile

```
# ---- Stage 1: builder ----
FROM python:3.12-slim AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /build
COPY requirements.txt .
RUN python -m venv /opt/venv \
    && . /opt/venv/bin/activate \
    && pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir -r requirements.txt

# ---- Stage 2: production ----
FROM python:3.12-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PATH="/opt/venv/bin:$PATH"

WORKDIR /app
COPY --from=builder /opt/venv /opt/venv

RUN groupadd --system appuser && useradd --system --gid appuser appuser
COPY --chown=appuser:appuser . .
USER appuser

EXPOSE 8000
CMD ["gunicorn", "-w", "2", "-k", "uvicorn.workers.UvicornWorker",
     "-b", "0.0.0.0:8000", "src.main:app"]
```

Docker Compose: контейнеры, сеть и тома

В файле docker-compose.yml описаны два сервиса. Сервис db использует официальный образ postgres:16-alpine, данные сохраняются в именованном томе postgres_data, а работоспособность проверяется командой pg_isready каждые десять секунд. Сервис app собирается из Dockerfile проекта, публикует порт 8000 и запускается только после того, как база перешла в состояние healthy.

Элемент	Назначение
services.db	Контейнер PostgreSQL 16 с томом для данных
services.app	Контейнер приложения, собранный из Dockerfile
restart: unless-stopped	Автоматический перезапуск после сбоя или перезагрузки
depends_on: condition: service_healthy	Запуск приложения только после готовности базы
volumes: postgres_data	Сохранение данных базы между перезапусками контейнеров

Элемент	Назначение
networks: app_net (bridge)	Внутренняя сеть, в которой приложение обращается к базе по имени db
ports: 8000:8000	Публикация порта приложения на хост

Листинг 20.2 — Фрагмент docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DATABASE_NAME:-library}
      POSTGRES_USER: ${DATABASE_USER:-library}
      POSTGRES_PASSWORD: ${DATABASE_PASSWORD:-library_password}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - app_net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DATABASE_USER:-library}"]
      interval: 10s
      timeout: 5s
      retries: 5


  app:
    build: .
    depends_on:
      db:
        condition: service_healthy
    environment:
      DATABASE_HOST: db
      USE_SQLITE: "false"
    ports:
      - "8000:8000"
```

Переменные окружения и конфигурация

Все параметры, различающиеся между средами, вынесены в переменные окружения и описаны в файле .env.example. Приложение читает их классом Settings на основе pydantic-settings, что обеспечивает проверку типов и значений при запуске: при отсутствии обязательного параметра контейнер завершается с понятной ошибкой, а не работает с неверной конфигурацией.

Переменная	Назначение
DATABASE_HOST, DATABASE_PORT	Адрес и порт сервера базы данных, внутри сети — имя сервиса db и порт 5432
DATABASE_NAME, DATABASE_USER, DATABASE_PASSWORD	Параметры подключения к базе
USE_SQLITE	Переключение между PostgreSQL и локальной SQLite для тестов

Переменная	Назначение
LOG_LEVEL	Уровень журналирования: INFO в эксплуатации, DEBUG при отладке
SECRET_KEY	Секретный ключ приложения, задаётся в среде и не хранится в репозитории

Файл .env исключён из репозитория через .gitignore, а в репозитории хранится только .env.example с безопасными значениями по умолчанию. Такой подход не допускает попадания паролей в систему контроля версий.

Health Check

Проверка работоспособности реализована на двух уровнях. Для базы данных используется встроенная команда pg_isready, которая подтверждает, что сервер принимает подключения. Для приложения настроена проверка маршрута /health командой Python: поскольку в образе python:3.12-slim отсутствует curl, запрос выполняется стандартной библиотекой urllib. Маршрут /health сам обращается к базе с простым запросом, поэтому его успешный ответ подтверждает не только запуск процесса, но и доступность данных.

Параметр start_period задаёт время прогрева, в течение которого неудачные проверки не учитываются, а interval, timeout и retries определяют частоту проверок и число допустимых ошибок до перевода контейнера в состояние unhealthy.

Листинг 20.3 — Проверка работоспособности приложения (src/api/health.py)

```
@router.get("/health", response_model=HealthOut)
def health(db: Annotated[Session, Depends(get_db)]) -> HealthOut:
    try:
        db.execute(text("SELECT 1"))
        database = "ok"
    except Exception:
        database = "unavailable"

    return HealthOut(status="ok" if database == "ok" else "degraded",
                     service="library-app", database=database)
```

Система логирования

Журналирование настраивается один раз при старте приложения функцией setup_logging(). Формат записи включает время, уровень, имя регистратора и

сообщение, а уровень берётся из переменной LOG_LEVEL. Отключение буферизации вывода в Dockerfile обеспечивает немедленное появление строк в журнале контейнера, поэтому команда docker compose logs показывает записи в реальном времени. Для диагностики инцидентов журнал содержит сведения о запуске приложения, инициализации базы и поступающих запросах с кодом ответа и временем обработки.

Конвейер CI/CD

Конвейер описан в файле .github/workflows/ci.yml и запускается при каждом push и pull request. Он состоит из трёх задач, выполняемых последовательно по зависимостям.

Задача	Инструменты	Содержание
lint	Ruff	Проверка стиля и типовых ошибок в каталогах src и tests
test	pytest, pytest-cov	Прогон тестов на SQLite с отчётом о покрытии и порогом --cov-fail-under=70
build	Docker Buildx	Сборка образа после успешных проверок, кэширование слоёв через GitHub Actions cache

Задача build запускается только при успехе lint и test, что не позволяет собрать и опубликовать образ с ошибками. Кэширование слоёв через cache-from и cache-to сокращает время повторных сборок. Использование SQLite в задаче test объясняется тем, что модульные и интеграционные тесты не должны требовать внешней базы; PostgreSQL проверяется при запуске через Docker Compose.

Проверка работоспособности системы

Сборка и запуск выполняются командами docker compose build и docker compose up -d. После запуска проверяются состояние контейнеров, доступность приложения и корректность работы с базой.

Проверка	Ожидаемый результат
docker compose build	Образ library-app:latest собран без ошибок, размер сокращён за счёт многоэтапной сборки

Проверка	Ожидаемый результат
docker compose ps	Контейнеры library_db и library_app в состоянии running, база отмечена как healthy
GET http://localhost:8000/health	Ответ со статусом ok и полем database = ok
docker compose logs app	Записи о запуске, подключении к базе и обработанных запросах
Остановка контейнера db	Приложение возвращает статус degraded, контейнер помечается нездоровым
Перезапуск стека	Данные сохраняются благодаря тому postgres_data
pytest --cov=src	Все тесты пройдены, покрытие выше установленного порога

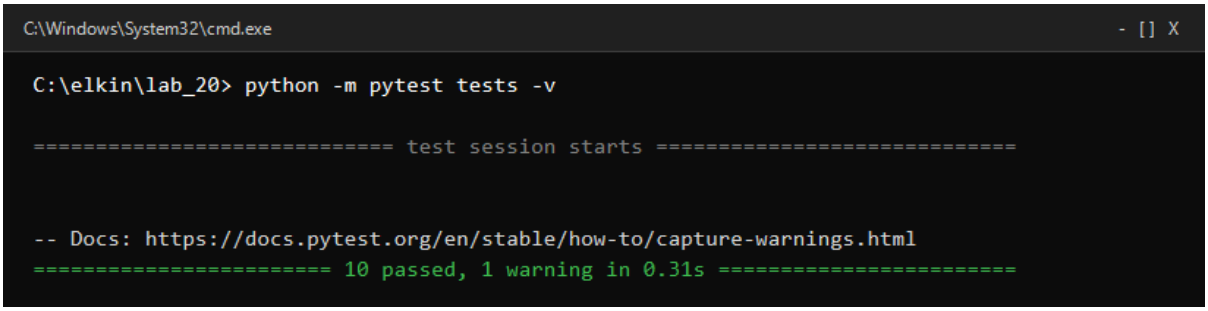


Рисунок 20.1 — Результат прохождения тестов приложения

Листинги ключевых фрагментов программы

Листинг 20.4 — Настройки приложения (src/config.py)

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    database_host: str = "localhost"
    database_port: int = 5432
    database_name: str = "library"
    database_user: str = "library"
    database_password: str = "library_password"
    use_sqlite: bool = False
    log_level: str = "INFO"
    secret_key: str = "change-me-in-production"

    @property
    def database_url(self) -> str:
        if self.use_sqlite:
            return "sqlite:///./library.db"
        return (f"postgresql+psycopg://{self.database_user}:"
                f"{self.database_password}@{self.database_host}:"
                f"{self.database_port}/{self.database_name}")
```

Листинг 20.5 — Конвейер GitHub Actions (.github/workflows/ci.yml)

```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
```

```
      with:
        python-version: "3.12"
    - run: pip install -r requirements.txt
    - run: ruff check src tests

test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - run: pip install -r requirements.txt
    - run: pytest --cov=src --cov-report=term-missing --cov-fail-under=70

build:
  needs: [lint, test]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: docker/setup-buildx-action@v3
    - uses: docker/build-push-action@v6
      with:
        context: .
        push: false
        tags: library-app:ci-${{ github.sha }}
```

Вывод

В ходе выполнения лабораторной работы приложение библиотеки подготовлено к развёртыванию и эксплуатации. Создан многоэтапный Dockerfile, который устанавливает зависимости в отдельном этапе, запускает приложение от непривилегированного пользователя и использует gunicorn с рабочими процессами uvicorn. Docker Compose описывает приложение и базу PostgreSQL с томом для данных, внутренней сетью и проверками работоспособности, а условие service_healthy гарантирует запуск приложения только после готовности базы. Конфигурация вынесена в переменные окружения, журналирование настроено единообразно, а конвейер GitHub Actions автоматически выполняет линтер, тесты с контролем покрытия и сборку образа, что позволяет обнаруживать ошибки до попадания изменений в эксплуатацию.